

Assignment 4.1: 8x16 Register File

Ziad Mostafa Abdelaziz

July 15, 2026

1 Introduction

A Register File consists of multiple operational registers configured within an array. This assignment implements an 8×16 Register File featuring 8 internal registers, each possessing a 16-bit width.

2 Block Interface & Specs

The Register File integrates a common address space for both read and write operations. The block module is defined without Verilog parameters utilizing the generic structure specifications given:

- **RdData (16-bit)**: Output for Read data bus.
- **WrData (16-bit)**: Input for Write data bus.
- **Address (3-bit)**: Shared address bus spanning exactly 8 registers.
- **WrEn (1-bit)**: Write Enable - allows overwriting the register on the clock edge.
- **RdEn (1-bit)**: Read Enable - latches register content to RdData.
- **CLK (1-bit)**: Clock trigger set on the positive edge.
- **RST (1-bit)**: Asynchronous active-low reset clearing all 8 registers simultaneously.

* Only one operation (Read or Write) is evaluated at any given time. * The Reset condition was initialized as the primary asynchronous block.

3 Simulation Waveforms

The Register file was validated with synchronous tests writing specific Hex logic ('h5A5A' and 'hC3C3') respectively into Register 3 and Register 6 and properly ensuring no overwriting occurs incorrectly.

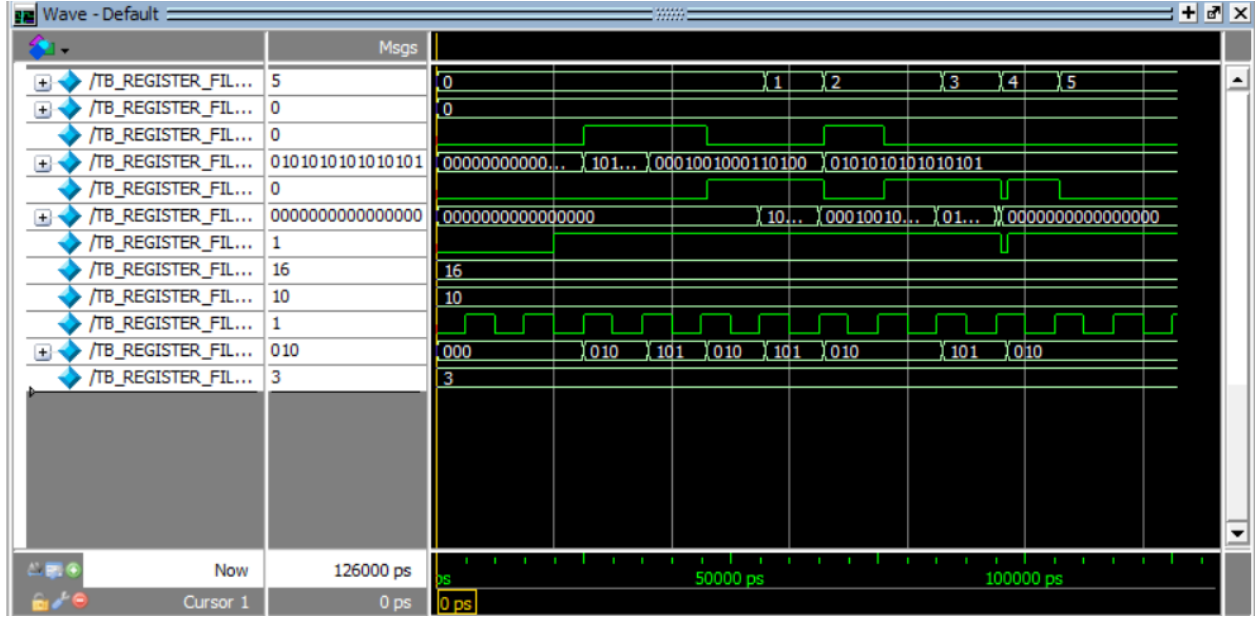


Figure 1: Register File Read/Write Operations Over Local Clock Edge

3.1 Waveform Analysis & Commentary

- **Reset Phase:** At $t = 0$, the RST signal is driven low (active-low assertion). This triggers the asynchronous reset path in the `always` block, forcing all eight registers (`Reg_File[0]` through `Reg_File[7]`) and the `RdData` output to `16'h0000`. In the waveform, all data buses remain at zero during this interval, confirming that the asynchronous clear operates independently of the clock edge.
- **Write Scenario 1 (Register 3 $\leftarrow$ 0x5A5A):** After RST is released high, `WrEn` is asserted with `Address = 3'd3` and `WrData = 16'h5A5A`. On the next rising edge of `CLK`, the value `0x5A5A` is latched into `Reg_File[3]`. In the waveform, we observe that `WrData` transitions to `5A5A` before the clock edge, and `WrEn` is high, satisfying the setup requirements. The data is stored internally and does not appear on `RdData` until explicitly read.
- **Write Scenario 2 (Register 6 $\leftarrow$ 0xC3C3):** Similarly, `WrEn` is reasserted with `Address = 3'd6` and `WrData = 16'hC3C3`. On the subsequent positive clock edge, the value is latched into `Reg_File[6]`. In the waveform, notice that `RdData` remains unchanged at `0x0000` during both write operations because `RdEn` is low — this confirms that only one operation executes at a time.
- **Read Scenario 1 (Register 3 $\rightarrow$ 0x5A5A):** `RdEn` is asserted with `Address = 3'd3`. On the rising clock edge, `RdData` updates to `0x5A5A`, matching the previously written value. The waveform shows the `RdData` bus transitioning from `0x0000` to `0x5A5A` exactly one clock cycle after the read command is applied.
- **Read Scenario 2 (Register 6 $\rightarrow$ 0xC3C3):** `RdEn` remains high while `Address` switches to `3'd6`. On the next clock edge, `RdData` properly updates to `0xC3C3`. This confirms that the previous write to Register 6 was correctly stored, and that reading a different register does not corrupt the output of Register 3 — each register maintains its own independent storage.

4 Simulation Testbench Logs

The resulting logged behaviors detail 4 specifically driven scenarios capturing writes and reads exactly onto designated registers (showing address tracking correctly outputs on ‘RdData’).

```
# =====
# REGISTER_FILE Testbench
# =====
# PASS | READ_REG2 | Address=2 -> RdData=43981
# PASS | READ_REGS | Address=5 -> RdData=4660
# PASS | READ_REG2_AFTER_OVERWRITE | Address=2 -> RdData=21845
# PASS | READ_REGS_UNCHANGED | Address=5 -> RdData=4660
# PASS | READ_REG2_AFTER_RESET | Address=2 -> RdData=0
# =====
# RESULTS: 5 PASSED, 0 FAILED
# =====
# ** Note: $finish      : E:/courses/Digital IC design Course/ASIC Flow/My Work/My Assignments/Assignment 4 Verilog/TB_REGISTER_FILE.v(111)
# Time: 126 ns  Iteration: 0  Instance: /TB_REGISTER_FILE
# 1
# Break in Module TB_REGISTER_FILE at E:/courses/Digital IC design Course/ASIC Flow/My Work/My Assignments/Assignment 4 Verilog/TB_REGISTER_FILE.v line 111
```

Figure 2: ModelSim Regfile Logging Output